Лабораторная работа № 2

Дисциплина: Аппаратные средства телекоммуникационных систем
Тема: Умножение больших матриц.
Выполнил: Валеев Дильюс Рафисович гр. 4404

Цель работы:

Освоить распараллеливание вычислительно тяжёлой задачи (перемножение квадратных матриц) на архитектуре с разделяемой памятью, научиться избегать состояний гонки за счёт разделения данных по потокам, применять модель fork-join и оценивать эффективность параллелизма через коэффициент ускорения. Выполнена задача 4 пособия «Параллельное программирование на OpenMP».

Результаты выполнения:

Программа запускалась с размером матрицы $N = 1000$ при числе потоков 2, 4 и 8. Команды запуска и вывод программы приведены на скриншотах.

```
C:\Users\Dilus>cd C:\Users\Dilus\source\repos\task4_LR2\x64\Debug

C:\Users\Dilus\source\repos\task4_LR2\x64\Debug>task4_LR2.exe 1000 2
Select input mode (1 - Custom range generation, 2 - Manual element input): 1
Enter min and max values for matrix elements (e.g., 1 100): 4 50

================================================
Matrix Multiplication Benchmark (Task 4 / Lab 2)
Matrix size          : 1000 x 1000
Available processors : 16
Requested threads    : 2
Value range          : [4.00, 50.00]
================================================
Sequential execution time        : 1.899205 sec
Parallel execution time (2 thr.) : 0.967139 sec
------------------------------------------------
Speedup                          : 1.96x
Verification                     : PASSED (OK)
================================================
```

Рис. 1 – Время при 2-х потоках.

```
C:\Users\Dilus\source\repos\task4_LR2\x64\Debug>task4_LR2.exe 1000 4
Select input mode (1 - Custom range generation, 2 - Manual element input): 1
Enter min and max values for matrix elements (e.g., 1 100): 4 50

================================================
Matrix Multiplication Benchmark (Task 4 / Lab 2)
Matrix size          : 1000 x 1000
Available processors : 16
Requested threads    : 4
Value range          : [4.00, 50.00]
================================================
Sequential execution time        : 1.889116 sec
Parallel execution time (4 thr.) : 0.519886 sec
------------------------------------------------
Speedup                          : 3.63x
Verification                     : PASSED (OK)
================================================
```

Рис. 2 -Время при 4-х потоках.

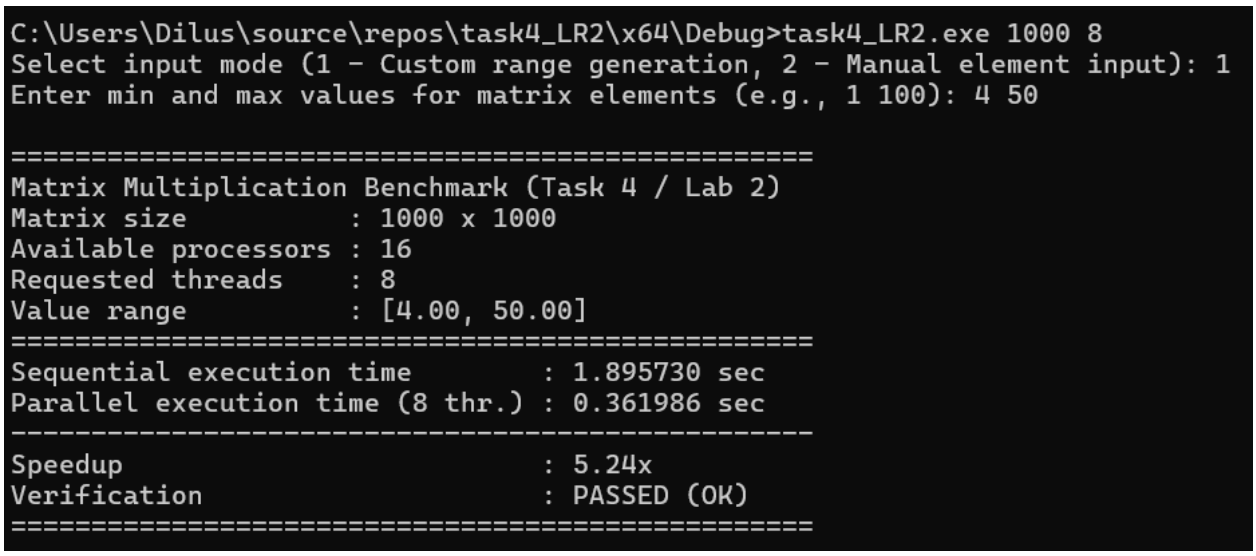


Рис. 3 – Время при 8-ми потоках.

Во всех запусках проверка verify_results вернула PASSED, что подтверждает корректность распараллеливания: параллельная версия вычисляет ту же матрицу, что и последовательная.

С ростом числа потоков время параллельного выполнения уменьшается, однако ускорение меньше числа потоков. Причины ускорения:

1. **Закон Амдала.** Последовательная доля кода (выделение памяти, заполнение матриц, проверка результата, ввод-вывод) не распараллеливается и ограничивает максимально достижимое ускорение.
2. **Накладные расходы на параллелизм.** Порождение потоков, распределение итераций и неявный барьер в конце parallel for требуют времени.
3. **Ограничение по памяти (memory-bound).** Во внутреннем цикле обращение к элементу $B[k][j]$ идёт по столбцу матрицы B, то есть с шагом N элементов в памяти. Это приводит к частым промахам кэша; несколько потоков одновременно обращаются к оперативной памяти, и её пропускная способность становится узким местом.
4. **Возможное превышение числа потоков над числом физических ядер** (если запрошено больше потоков, чем доступно процессоров) приводит к разделению ядер между потоками и переключениям контекста.

Следует отметить, что при малом размере матрицы (например, $N < 200$) параллельная версия может оказаться медленнее последовательной из-за того, что накладные расходы на порождение потоков превышают выигрыш от распараллеливания. Поэтому для бенчмарка выбран размер $N = 1000$.